Vox Scripta

Technical status report

1 November 2026 · version 0.3.0, build 50 · commit `892df7b`

A local-first voice-to-text application for Apple Silicon, now being taken to five platforms. This report states what the goal is, measures the project against it, and describes in detail the technology and the code that exist today — including what does not work and what has not been verified.

Everything quantified here was measured on the development machine during the work described, not estimated. Where a claim has not been tested, the report says so rather than rounding it up.

1 The goal

One application, genuinely the same application, running on macOS, iOS, Android, Windows and Linux — that turns recorded speech into an accurate, readable transcript entirely on the user's own device.

That sentence has two halves and both are load-bearing. *The same application* means a recording made on one platform opens on another, the same rules shape the transcript, and a feature added once appears everywhere. *Entirely on the user's own device* is not a preference; it is the product.

The constraints that follow from it

Constraint	Where it comes from	Status
No cloud service of any kind — no OpenAI, Google, AWS, Azure, Deepgram, or any hosted transcription or LLM API	§26 and the original brief	**held**
No required internet connection for core functionality	Original brief	**held**
The original audio file is never deleted, modified, overwritten or trimmed	§5	**held**
Silence detection uses deterministic signal processing, never a model	§6	**held**
No telemetry, no uploads, no remote processing	§26	**held**
No hardcoded credentials or API keys	§27	**held**
Architecture before code; no placeholders presented as finished	§37, §38	**held**

The privacy constraint is enforced structurally rather than by intention. The macOS application ships with no network entitlement at all, and the build script fails if one ever appears:

```
if codesign -d --entitlements - "$APP" | grep -q "network.client"; then
    echo "FAIL: the bundle carries a network entitlement"; exit 1
fi
```

The iOS `Info.plist` carries a microphone key and a background-audio key and nothing else — no network key, no location, no camera. Models are fetched at *build* time by scripts in `Tools/`; the application itself never reaches the network.

2 Scorecard against the goal

Five platforms, five stages each. The last stage is the one this project keeps learning the hard way: code that compiles and code that works are different claims.

Platform	Engine	Interface	Packaged	Runs	Proven on hardware
macOS	yes	yes	yes	yes	yes
iOS	yes	yes	yes	simulator	no
Linux	6 of 7 modules	no	no	no	no
Windows	not attempted	no	no	no	no
Android	not attempted	no	no	no	no

Windows and Android are marked *not attempted* rather than *failing*. Nothing has been built for them yet, but the work that was done for Linux — replacing six Apple frameworks behind the interfaces that already existed — is most of what they need too. The build scripts already take an Android ABI as an argument.

The honest headline

The engine is close to done. The applications are not. Four of the five platform-specific subsystems that stood between this project and its goal have been replaced and measured; six of seven Swift modules cross-compile for Linux. What does not exist is an interface on anything but Apple, and nothing has ever run on a real phone.

3 How the system is put together

The project is fourteen Swift modules, five C shims and a C++ core, totalling **17,620 lines of source** and **7,958 lines of tests**. The shape matters more than the size: the engine is a stack of narrow protocols, and every platform-specific thing sits behind one of them. That is why the port turned out to be a substitution rather than a rewrite.

Module	Lines	What it does	Portable?
VoxScriptaCore	1,740	Time, identifiers, transcript model, errors, tokenizer, paragraphs	yes
VoxScriptaDSP	2,533	Decode, frame analysis, silence detection, chunk planning, recording	yes
VoxScriptaTranscription	1,426	Engine protocol, Apple and Whisper engines, scheduler, merger	yes
VoxScriptaLanguage	2,777	CLEAN rules, POLISH, the validation gauntlet, model hosts	yes
VoxScriptaProject	1,289	Project packages, manifests, job store, atomic writes, resume	yes
VoxScriptaPipeline	832	The job graph that drives all of the above	MLX dependency
VoxScriptaExport	621	txt, md, docx, srt, vtt	yes
VoxScriptaUI	3,676	The SwiftUI interface, shared by macOS and iOS	Apple only
VoxScriptaMLX	225	Bundled-model host, Apple Silicon	Apple only
voxscripta-cli	802	Command line: analyze, transcribe, bench, wer, run, export	yes

The four seams that made the port possible

Each of these is a small protocol written long before the port was contemplated. Replacing a platform meant writing one conforming type, not touching the pipeline.

Seam	Shape	Apple	Everywhere else
`TranscriptionEngine`	4 methods: capabilities, prepare, transcribe, release	SpeechAnalyzer	whisper.cpp
`LanguageModelHosting`	5 members: configuration, isReady, load, rewrite, unload	FoundationModels / MLX	llama.cpp
`CanonicalDecoding`	2 methods: probe, decode	AVFoundation	FFmpeg
Recording	start, pause, resume, finish, cancel	AVCaptureSession	miniaudio + FFmpeg

The canonical format, and why it made everything easier

Every source file is decoded exactly once into a headerless raw stream of **16 kHz mono 32-bit float** samples, which every later stage memory-maps rather than decoding again. This was designed for memory behaviour — a six-hour recording must cost no more RAM than a six-second one — but it paid off twice during the port:

• Whisper's only accepted input is 16 kHz mono float. Feeding it a chunk is a seek and a read, with no resampling and no audio framework involved, which is why that reading code ports unchanged.
• A format with no header and one fixed layout needs no parser, so the code that reads it is arithmetic rather than a dependency.

4 What was built: recording

The application could only transcribe files it was given. On a phone that is the wrong shape entirely — the audio someone wants transcribed is usually either already in Files or has not been recorded yet. Recording was added, and the empty state that said *“Drop an audio file to begin”* — a Mac instruction with no meaning on a phone — was replaced.

4.1 Crash-safe recording

The first implementation used `AVAudioRecorder`, which writes the MPEG-4 index only when stopped cleanly. Anything else — a crash, a force quit, a flat battery — left a file full of audio that nothing could open. This was measured, not hypothesised: after force-quitting the app mid-recording, 2.4 MB of audio reported no duration at all.

For a feature whose purpose is recording a two-hour lecture, losing the lecture because the app died at minute 119 is not an acceptable failure mode. The writer was rebuilt on `AVAssetWriter` with `movieFragmentInterval` set to two seconds: the index is written as it goes, so a file that is never finalized is still playable up to the last completed fragment.

Test	Recovered	Meaning
Fragmented writer, SIGKILL 9 s in	**8.03 s**	Only the fragment in flight was lost; this project's own decoder opened the file and found 8.010 s
Fragmented writer, abandoned at 6 s	**4.04 s**	Portable path, same mechanism
Unfragmented control, abandoned at 6 s	**0.00 s**	The control proves the mechanism rather than a coincidence

The control case matters. Without it the test would show only that a file could be read, not that fragmenting was what made it readable.

4.2 Defects found by running it rather than reading it

Defect	How it presented	Fix
Capture start blocked the main thread	On a simulator whose input device will not open, `record()` never returned. The app stopped drawing; the Record button looked simply dead.	Start moved off-main, raced against an 8-second deadline through a lock and a continuation. A task group cannot do this — it waits for every child before exiting.
A running session is not a working one	The capture session started, posted a runtime error a moment later, and delivered nothing. The recorder believed it was recording silence.	`start()` now waits for the first sample buffer. That is the only evidence audio exists.
Capture interruptions were unobserved	A phone call would end a two-hour recording with no message.	AVAudioSession and AVCaptureSession interruptions are observed, reported by reason, and resumed automatically.
A data race the compiler could not see	`writer` and `input` are mutated on the capture queue and were touched from the main actor. The class is `@unchecked Sendable`.	Both now hop onto that queue, which also lets in-flight buffers land before the writer closes.
The meter read full scale at rest	Initialised to 0, which in dBFS is *pegged*, not silent.	Starts at the floor.
An Int where a Double belonged	Inside a `[String: Any]` literal every value gets `Any` as its contextual type, so the integer literal in a ternary stayed an Int and the fallback sample rate reached AVFoundation as `Int(44100)`.	Typed locals. Caught by a new test, not by review.
No file dialogs were ever presented	The model kept the state when AppKit panels were replaced and no view consumed it — so Open, Download and Export were buttons that set a flag. On macOS too.	All three dialogs wired up.

5 What was built: the confidence floor

Recording in the app made an old behaviour impossible to ignore. Asked to transcribe room tone, the recogniser does not return nothing — it returns confident-looking sentences that were never said. Four seconds of an empty room before the speaker started produced:

```
“George, you kill the state! Why, sister, stop, stop.”
```

Importing a file rarely hit this, because a file usually starts where the speech does. A recording starts when you press the button and ends when you reach for Stop, so every transcript now has silence at both ends.

The threshold was measured, not chosen

Every segment from six recordings — two read aloud, two conversational, one Spanish monologue, one made in the app — was collected with its confidence. 67 segments. The two populations do not overlap:

Population	Lowest	Highest	Example
Real speech	**0.8168**	0.9922	“Um, we did not approve it.” — a mumbled aside
Invented from noise	0.1100	**0.5800**	“Serantella!”

Nothing landed between 0.58 and 0.8168. The floor sits at **0.65**: inside the empty band and deliberately nearer the noise, because a false line costs a moment's reading and a lost real one costs something the listener never learns was said.

Where it lives, and why that matters

In the CLEAN stage, which is where deterministic edits live and where an audit log already exists. So the verbatim transcript keeps every word, `clean.txt` loses the invented ones, and `clean_edits.json` records each drop with its score and the floor it failed:

```
confidence 0.1944 below floor 0.65    George, you kill the state!
confidence 0.1100 below floor 0.65    Why, sister, stop, stop.
confidence 0.4850 below floor 0.65    talk to me.
confidence 0.5800 below floor 0.65    Serantella!
confidence 0.4650 below floor 0.65    this.
```

Nothing is removed that cannot be read somewhere else — the same bargain the rest of CLEAN makes. Two details worth recording: scores are compared as `Float`, the width the engine reports them in, because widening first turns 0.65 into 0.6499999761581421 and a segment exactly on the floor would fall through it; and a segment with no reported confidence is kept rather than discarded on a missing number.

A structural test was added at the same time, because this codebase had already shipped the bug once: every `CleanParameters` field must reach the stage hash, counted against the type's own field count, so a new parameter cannot be added without one. A parameter missing from the hash is a setting that silently does nothing. The guard was verified by removing the floor from the description and watching it fail.

6 The port: four backends, each measured

Three decisions were taken before any code: SwiftUI stays on Mac and iPhone with one Flutter interface for the other three; Apple's engines stay on Apple with portable replacements elsewhere; the engine is proven on a desktop before anything is packaged.

6.1 A finding that reversed the plan of record

`Package.swift` had said since the Android spike that the C++ port would replace the Swift module by module, with a parity test target holding the two to the same answers. Counting before following it showed that was the expensive road:

Already free of Apple frameworks	Lines
Core, Project, Pipeline, Export, CLI, Bench	**5,484**
Non-portable Foundation calls found across all of them	**8**

Those 5,484 lines are the job graph, the project format, the resume planner, every exporter and the command line. Against that, porting to C++ meant moving roughly 8,000 working, tested lines into a second implementation that would then have to be kept agreeing forever — which is precisely why that parity target had to exist. Swift builds for Linux, Windows and Android, so the recommendation was to keep one Swift engine and swap four backends. The 568 lines of C++ keep their value: they proved the NDK toolchain, the four-ABI cross-compile and the JNI contract.

6.2 Recognition — whisper.cpp

The second engine had been a stub that throws since Phase 0. It is now whisper.cpp, linked as a static library through a header shim, conforming to the same four-method protocol as Apple's. Measured on the same three passages with the same word-error-rate harness:

Engine	Word error rate
Apple SpeechAnalyzer	5.56%
whisper.cpp, ggml-base	**2.50%**

The smaller, portable engine read this material *better* than the one shipping today. That is one fixture and not a reason to switch anything, but it retires the assumption that the portable path is the compromise.

One finding recorded at the point it matters: Whisper's token probabilities are not Apple's confidence values, so the CLEAN confidence floor — measured at 0.65 against Apple's numbers — must be recalibrated before it is trusted against this engine.

6.3 Polishing — llama.cpp

POLISH is the only stage that can change what a sentence means, which is why it sits behind an eight-check validation gauntlet. A second backend therefore is not proven by producing text; it is proven by producing text the existing gauntlet accepts. Running Qwen2.5-1.5B-Instruct — deliberately the same model the MLX backend bundles, so any difference is the backend and not the model:

```
  we sold five hundred units in the third quarter
+ We sold five hundred units in the third quarter.
  david met with sarah chen on tuesday
+ David met with Sarah Chen on Tuesday.
```

The gauntlet accepts the output, Spanish comes back in Spanish with its negation intact, and the same sentence twice gives the same text. Cardinality is true by construction — one call per sentence with a fresh context — rather than enforced by a schema the backend must support. Decoding is greedy rather than temperature zero, which is the stronger guarantee the provenance already claims.

The prompt, token budget and output cleaning were moved out of the MLX host into a shared `LocalPrompting` that both backends call, with a test asserting they stay shared. Two backends with two prompts would be two behaviours to validate, and the difference would surface as a transcript that reads differently on a phone than on a PC for no reason anyone chose.

6.4 Decoding — FFmpeg

The first attempt used miniaudio, which reads wav, flac and mp3 and cannot read AAC. That was not a rounding error in coverage: **the app records m4a**, so a recording made on a Mac could not have been opened on Linux, and importing audio from video would not have worked either. Three options were put forward — FFmpeg, a system decoder per platform, or recording WAV off Apple and accepting the import gap — and FFmpeg was chosen.

File	Offset from AVFoundation	Worst difference
WAV	0 samples	**0.0000138**
m4a (AAC)	0 samples	**0.0000415**

About one part in seventy thousand, measured on the same files by the parity test the miniaudio version introduced. That test had already earned its keep: it failed first for miniaudio by 0.076, which turned out to be one sample of group delay between two resamplers — at 440 Hz a period is 36 samples at the canonical rate, so comparing index against index was measuring the delay instead of the audio.

On the licence

FFmpeg is built with `--disable-gpl --disable-nonfree --disable-version3`, which makes it LGPL v2.1, and as *shared* libraries. That combination is what keeps the obligation satisfiable without publishing this application's source: the libraries ship beside the binary and can be replaced. Static linking would also be possible but obliges handing out object files for relinking, which is a worse deal for no benefit. Cut down to only the demuxers and decoders AVFoundation reads, the build is **2.9 MB** rather than the tens of megabytes a full FFmpeg is.

Two bundling problems were found by checking rather than reasoning. The linker wrote absolute build-directory paths into the app binary, so the installed app would have launched only on the machine that built it and only until that directory was cleaned; the libraries are now copied into `Contents/Frameworks`, re-pathed to `@rpath` and signed before the app is. And a grep for sibling dependencies exits 1 for libavutil, which has none — with `pipefail` that silently killed the build script after the localization step.

6.5 Recording off Apple — miniaudio and FFmpeg

miniaudio opens the capture device — WASAPI, ALSA or PulseAudio, AAudio, CoreAudio — behind one API, and FFmpeg's AAC encoder and mov muxer turn what it delivers into **the same fragmented m4a the Apple recorder writes**. FFmpeg was already linked; enabling its encoder and muxer cost 400 KB and bought a great deal: a recording made on Android opens on a Mac, the crash-safety mechanism is one mechanism rather than two, and nothing downstream has a second format to learn. FFmpeg's own AAC encoder, so the build stays LGPL — libfdk would not have.

Two consequences fell out of doing it this way. Pausing needs no timeline surgery, unlike on Apple: nothing is written while paused, so there is no gap to skip. And the partial frame at the end is padded rather than dropped — AAC takes whole 1024-sample frames, and quietly losing up to 23 ms off the end of every recording is the kind of thing nobody notices until the timings stop lining up.

7 Swift on Linux

Cross-compiled from the Mac using the swift.org 6.4.0 toolchain and the Static Linux SDK. Xcode's Swift and swift.org's are different builds of the same version number, and the SDK's prebuilt modules will not load into the wrong one — which presents as a confusing *“compiled module was created by a different version of the compiler '”*.

Module	Linux	Note
VoxScriptaCore	**builds**	
VoxScriptaExport	**builds**	
VoxScriptaProject	**builds**	Vendored SQLite, swift-crypto
VoxScriptaLanguage	**builds**	POLISH weaker — see below
VoxScriptaDSP	**builds**	Portable FFT
VoxScriptaTranscription	**builds**	whisper.cpp by default
VoxScriptaPipeline	**blocked**	MLX dependency, not its own code

The estimate of eight call sites was close but wrong

What broke	Why	What was done
`OSLog`	Missed by the original survey, which grepped a fixed list of frameworks.	A silent stand-in, not a `print`. The Apple logger marks user content `.private`; a substitute that printed would break that promise on the platforms least able to notice.
`String(localized:)`	Apple-only Foundation.	A gated shim returns the English already at the call site. The engine therefore speaks English off Apple — a real limitation. The fix is for the Flutter interface to translate from the error *case*, which carries the facts, not the rendered sentence.
`SQLite3`	A module Apple ships and nothing else does; the static Linux SDK has no libsqlite3 at all.	The official amalgamation is vendored and compiled for every target, verified against the SHA3-256 sqlite.org publishes.
`CryptoKit`	Apple only.	swift-crypto mirrors the API and was already in the dependency graph.
`F_FULLFSYNC`	Darwin-only, and only needed because macOS's `fsync` does not flush the drive cache.	Linux's does, so plain `fsync` is the equivalent rather than a downgrade.
Bookmarks and the Trash	The two that were predicted.	Off Apple the source reference is the path, stored in the same field, and a deleted project moves to a `Deleted` folder beside the library so deletion stays recoverable.
Mach memory probe	`phys_footprint` has no Linux equivalent.	VmRSS from `/proc/self/status` — which *does* count the mapped canonical audio that phys_footprint was chosen to exclude. Weaker, and the code says so rather than equating them.

A verification detail worth keeping

SQLite publishes **SHA3-256**, not SHA-256. The first verification “failed” and looked exactly like a tampered download. It was not — the wrong algorithm had been used. Verified correctly before anything was vendored.

8 The portable FFT, in detail

Accelerate was the last framework standing between DSP and Linux. The surface is small — Hann window, RMS, min/max, elementwise multiply — but it includes a real FFT that feeds the spectral-flatness feature, and silence detection's thresholds were calibrated against it with measured accuracy claims attached.

Compiled everywhere, so it can be checked somewhere

`PortableDSP` is compiled on every platform rather than only off Apple. That is the point: a parity suite runs it against vDSP on the same input, so the version Linux depends on is the version that was measured. The alternative is code that only ever executes where nobody can check it.

Primitive	Agreement with Accelerate
Power spectrum	< 1e-4 relative, worst bin, sizes 256 / 512 / 1024
Spectral flatness	< 1e-5 absolute — the number silence detection reads
Hann window	6e-8 absolute
RMS, min/max, elementwise multiply	exact

Three vDSP conventions had to be reproduced, not improved on

A textbook FFT would have been quietly wrong in a way that moved every boundary this project has measured:

1. vDSP's real FFT returns *twice* the mathematical transform.
2. It packs the two purely real bins together — DC in `realp[0]`, Nyquist in `imagp[0]`.
3. `vDSP_zvmags` therefore reports the sum of their squares as the first “bin”, rather than the DC power alone.

The implementation is an iterative radix-2 Cooley-Tukey transform in Double, on a half-length complex sequence with the standard untangling — iterative so a long recording's frames cost no stack, and in Double because the untangling subtracts nearly equal numbers where Float would lose exactly the quiet bins that spectral flatness is most sensitive to.

The round-trip test failed first, and it was the test rather than the transform: the swap identity `IFFT(X) = swap(FFT(swap(X)))/N` leaves the real part in the first array, and it was reading the second.

9 The technology, and why each piece is there

Component	Role	Licence	Size
Apple SpeechAnalyzer	Recognition on Apple platforms	System	—
Apple FoundationModels	POLISH on Apple platforms	System	—
MLX + Qwen2.5-1.5B-4bit	POLISH fallback on Apple Silicon	MIT	835 MB
whisper.cpp (pinned)	Recognition everywhere else	MIT	44 MB src
llama.cpp (pinned)	POLISH everywhere else	MIT	183 MB src
FFmpeg n7.1.1	Decode and mux, all non-Apple platforms	**LGPL v2.1**	2.9 MB built
miniaudio 0.11.25	Capture device, all non-Apple platforms	Public domain	13 MB src
SQLite 3.54.1 amalgamation	Job store on every platform	Public domain	9.9 MB
swift-crypto	SHA-256 where CryptoKit is absent	Apache 2.0	—
ggml-base.bin	Whisper model (build-time fetch)	MIT	141 MB
Qwen2.5-1.5B GGUF	POLISH model (build-time fetch)	Apache 2.0	1.0 GB

One deliberate consolidation: whisper.cpp and llama.cpp each vendor their own copy of ggml, and linking both is a duplicate-symbol error. llama.cpp is built and installed first and whisper.cpp built against that install with `WHISPER_USE_SYSTEM_GGML` — one ggml in the binary, one set of backends, one place a GPU is chosen.

10 Verification

442 tests across ten targets, 7,958 lines of test code against 17,620 lines of source. Every number quoted in this report came from one of them or from a measurement taken on the development machine.

Target	Tests	Covers
VoxScriptaCore	68	Time, identifiers, transcript model, paragraphs, errors
VoxScriptaDSP	96	Decode parity, silence ground truth, chunking, FFT parity, recording
VoxScriptaLanguage	112	CLEAN invariants, the gauntlet, Spanish corpus, confidence floor

Target	Tests	Covers
VoxScriptaTranscription	49	Engines, scheduler, merger, word error rate, Whisper
VoxScriptaProject	32	Packages, atomic writes, stage invalidation, §5 deletion
VoxScriptaAppTests	27	Settings, progress, recording transport, crash safety
VoxScriptaExport	21	Five formats, subtitle timing, DOCX validity
VoxScriptaPipeline	20	Crash recovery, cancellation, end-to-end CLEAN
VoxScriptaCoreParity	12	C++ core against the Swift original
VoxScriptaMLX	5	Bundled model loads and rewrites

The pipeline target's 20 tests have not been re-run since the port work. They depend on Apple's on-device language model, which was responding so slowly on this machine that a single run exceeded 45 minutes without completing. That target depends only on `VoxScriptaPipeline` and `VoxScriptaTestSupport`, neither of which the port changed, but it is an unticked box and is recorded as one.

Accuracy, measured on real audio

Measurement	Result	Material
Spanish word error rate	1.33%	Synthesized speech
Spanish word error rate	6.33%	Human LibriVox reading
English word error rate, Apple	5.56%	Three synthesized passages
English word error rate, whisper.cpp	2.50%	The same three passages
Transcription throughput	~120×	Faster than real time
POLISH throughput	~1.4×	Of audio duration

11 What is not done

Measured against the goal — the same application on five platforms, entirely on-device — this is the remaining distance.

Gap	Consequence	Size
No interface on Android, Windows or Linux	Three of the five platforms have no application at all. The Flutter interface has not been started.	Large
Nothing has run on a real phone	iOS builds and runs in the simulator only. Background recording, interruptions and the microphone itself are untested on a device. Needs a signing identity.	Medium
Pipeline does not cross-compile	It depends on VoxScriptaMLX, which pulls in a HuggingFace package that will not build against musl — and SwiftPM builds that dependency even when it is marked Apple-only. The fix is to hand Pipeline a host rather than let it resolve one.	Small
Model sizes are unresolved	835 MB MLX model, 141 MB Whisper model, 1.0 GB GGUF. The installed macOS app is 924 MB. None of this can ship in a phone application as it stands.	A decision
POLISH is weaker off Apple	Semantic distance abstains, named entities fall back to capitalized tokens, content words fall back to surface forms. English loses the second of its two defences against a reversed meaning.	A decision
Translation does not exist	`TranscriptionOptions.translate` is declared and ignored.	Small
Android C++ core is a spike	568 lines cross-compile for four ABIs and pass 13 checks; nothing has been executed on a device. It is no longer the plan of record.	Superseded

12 Decisions waiting on you

These are not tasks. Each one changes what gets built next, and none can be settled from the code.

12.1 Which models ship where

This is now blocking rather than theoretical, because the Flutter packaging step needs the answer. Three model files total 2 GB and the installed macOS application is 924 MB. A phone application cannot carry that. The options are a smaller model, a download after install, or relying on the platform's own model and accepting that POLISH is unavailable when it is absent — and the answer may differ per platform.

12.2 Whether POLISH runs off Apple at all

English had two independent defences against a reversed meaning: the lexical polarity count and the semantic-distance check. Off Apple only the first survives — which is exactly the position Spanish has always been in, and the Spanish rejection corpus measured that lexical checks alone caught every polarity inversion. That evidence does not exist for English. Before POLISH is enabled on Android, Windows or Linux, the English corpus should be run with the embedding disabled, the way `PolishReadiness` already gates by measurement rather than assertion.

12.3 An iOS signing identity

Until there is one, nothing can be installed on a phone, and a whole class of behaviour — background recording, call interruptions, the real microphone — stays unverified. This is paperwork rather than engineering, but it gates real knowledge.

A recommendation on sequence

The engine work is substantially done and each piece is measured. The cheapest next step that converts work into something you can hold is **inverting the Pipeline dependency** — small, well understood — which unblocks the command line on Linux. A working Linux CLI would be the first time this project runs end to end on a non-Apple platform, and it needs no interface, no store and no signing identity to prove it.

Appendix Commits in this period

Commit	Subject
`892df7b`	A portable FFT, measured against Accelerate rather than against theory
`3616d80`	Four modules build for Linux, and the FFT is what is left
`f21a949`	Recording off Apple, writing the same file Apple writes
`712a9dd`	FFmpeg for decoding, so every platform reads every file
`da08870`	Decoding without AVFoundation, and the gap it leaves
`c4a006c`	POLISH without Apple: llama.cpp behind the same contract
`ca88026`	A real Whisper engine, measured against Apple's on the same audio
`8f3eee5`	Start the port: decide the shape, vendor whisper.cpp
`259c7d9`	Keep recording when the app is not in front
`6ae0994`	Make “recording” mean audio is arriving, and show it on iOS
`c1af927`	Write recordings in fragments, so a crash cannot lose one
`66abf0f`	Halve the recording screen's redraw rate
`9bca8c8`	Bump to 0.3.0
`cb78a04`	Drop sentences the engine invented out of silence
`830bc9c`	Record in the app, and stop telling a phone to drop a file

Fifteen commits. Every one of them is reproducible from the repository, and the measurements quoted throughout this report are in the commit messages and the test output rather than in anyone's memory.